ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

Aula 9

Sumário

- Representação de números inteiros
  - Complemento para 2
- Excesso de capacidade de representação (Overflow)

Representação de números inteiros

Conjunto dos números inteiros

- A subtracção de dois números $(\mathrm{A} - \mathrm{B})$ em que $\mathrm{A} < \mathrm{B}$ não é representável em $\mathbb{N}$
- No entanto, no conjunto $\mathbb{Z}$ (inteiros), para cada número existe o seu simétrico
  - Representando o simétrico de x por $-x$ (representação por sinal e magnitude)
    - $x \in \mathbb{Z}$ $(-x) \in \mathbb{Z}$ de tal forma que $x + (-x) = 0$

Representação dos números inteiros

- Em sistemas digitais, existem algumas restrições a ter em conta na representação de números:
  - A quantidade de dígitos disponíveis para representar um número é finito
  - O sinal, tal como os restantes dígitos, terá de ser codificado usando um dos dois símbolos disponíveis (0 ou 1)
  - O código a usar para representar os números inteiros deve facilitar (reduzir a complexidade) das operações aritméticas básicas a serem implementadas em hardware

Representação dos números inteiros

- Em sistemas digitais, existem várias formas para representar os números negativos
  - Sinal e magnitude (simétrico)
  - Complemento para 1 (simétrico)
  - Complemento para 2 (assimétrico)
  - Offset (assimétrico)
  - Base -2 (assimétrico)

- Algumas das formas de representação (e.g. Sinal e magnitude, complementos) utilizam um bit adicional, o bit mais significativo, para representar o sinal (‘0’ para +, e ‘1’ para -)

Representação dos números inteiros

- Em sistemas digitais, existem várias formas para representar os números negativos
  - Sinal e magnitude (simétrico)
  - Complemento para 1 (simétrico)
  - Complemento para 2 (assimétrico)
  - Offset (assimétrico)
  - Base -2 (assimétrico)

- Algumas das formas de representação (e.g. Sinal e magnitude, complementos) utilizam um bit adicional, o bit mais significativo, para representar o sinal (‘0’ para +, e ‘1’ para -)

Representação dos números inteiros

- Sinal e magnitude
  - O bit mais significativo representa o sinal e os restantes bits representam o seu valor absoluto
    - Ex: $-3_{(10)}=1011_{(2)}$
  - O valor zero tem duas representações...

Decimal	Sinal e magnitude
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	1000
-1	1001
-2	1010
-3	1011
-4	1100

Representação dos números inteiros

- Sinal e magnitude
  - O bit mais significativo representa o sinal e os restantes bits representam o seu valor absoluto
    - Ex: $-3_{(10)}=1011_{(2)}$
  - O valor zero tem duas representações...

Decimal	Sinal e magnitude
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	1000
-1	1001
-2	1010
-3	1011
-4	1100

bit de sinal

Representação em complementos

- A forma mais usada para representar os numeros inteiros nos sistemas digitais actuais é denominada por código de complementos
- O código de complementos é baseado no pressuposto de que:
  - os operandos e os resultados operados por um sistema digital têm sempre o mesmo número de dígitos
  - encontrar o simétrico de um número é encontrar uma configuração que somada com ele seja igual a zero para o número de dígitos estabelecido
  - O código dos complementos é modular, em que o módulo é igual a 2^n

Complemento para 1

- Complemento para 1 (complemento restringido)
  - O complemento para 1 de N, em n bits, é definido como $(2^n - 1) - N$
  - $2^n - 1$ = 11..1111 (n bits '1')
  - Complementar para 1 corresponde a inverter todos os bits
    - Ex: $-3_{(10)} = -(0011_{(2)}) = ((1111 - 0011)_{(2)})_{(C'1)} = 1100_{(C'1)}$
  - Representação
    - Números positivos – representados pelo seu módulo (magnitude)
    - Números negativos – representados pelo complemento para 1
    - O valor zero tem duas representações...

Decimal	Complemento para 1
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	1111
-1	1110
-2	1101
-3	1100
-4	1011

Complemento para 1

- Complemento para 1 (complemento restringido)
  - O complemento para 1 de N, em n bits, é definido como $(2^n - 1) - N$
  - $2^n - 1$ = 11..1111 (n bits '1')
  - Complementar para 1 corresponde a inverter todos os bits
    - Ex: $-3_{(10)} = -(0011_{(2)}) = ((1111 - 0011)_{(2)})_{(C'1)} = 1100_{(C'1)}$
  - Representação
    - Números positivos – representados pelo seu módulo (magnitude)
    - Números negativos – representados pelo complemento para 1
    - O valor zero tem duas representações...

Decimal	Complemento para 1
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	1111
-1	1110
-2	1101
-3	1100
-4	1011

bit de sinal

Complemento para 2

- Complemento para 2 (complemento verdadeiro)
  - O complemento para 2 de N, em n bits, é definido como:
    - $(2^n) - N$, para $N \neq 0$
    - 0, para $N = 0$
  - Complementar para 2 corresponde a complementar para 1 (inverter todos os bits) e somar 1
    - Ex: $-3_{(10)} = -(0011_{(2)}) = (1100_{(C'1)} + 1_{(2)})_{(C'2)}$
      $= 1101_{(C'2)}$
  - Representação
    - Números positivos – representados pelo seu módulo (magnitude)
    - Números negativos – representados pelo complemento para 2
    - Extensão para mais bits de representação é obtida por extensão do bit de sinal (bit mais à esquerda)
    - Uma única representação para o valor zero!

Decimal	Complemento para 2
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	---
-1	1111
-2	1110
-3	1101
-4	1100

Complemento para 2

- Complemento para 2 (complemento verdadeiro)
  - O complemento para 2 de N, em n bits, é definido como:
    - $(2^n) - N$, para $N \neq 0$
    - 0, para $N = 0$
  - Complementar para 2 corresponde a complementar para 1 (inverter todos os bits) e somar 1
    - Ex: $-3_{(10)} = -(0011_{(2)}) = (1100_{(C'1)} + 1_{(2)})_{(C'2)}$
      $= 1101_{(C'2)}$

complemento para 1 de $-3_{(10)}$

  - Representação
    - Números positivos – representados pelo seu módulo (magnitude)
    - Números negativos – representados pelo complemento para 2
    - Extensão para mais bits de representação é obtida por extensão do bit de sinal (bit mais à esquerda)
    - Uma única representação para o valor zero!

Decimal	Complemento para 2
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	---
-1	1111
-2	1110
-3	1101
-4	1100

bit de sinal

Complemento para 2

- Complemento para 2 (complemento verdadeiro)
  - O complemento para 2 de N, em n bits, é definido como:
    - $(2^n) - N$, para $N \neq 0$
    - 0, para $N = 0$
  - Complementar para 2 corresponde a complementar para 1 (inverter todos os bits) e somar 1
    - Ex: $-3_{(10)} = -(0011_{(2)}) = (1100_{(C'1)} + 1_{(2)})_{(C'2)}$
      $= \boxed{1101_{(C'2)}}$

complemento para 2 de $-3_{(10)}$

  - Representação
    - Números positivos – representados pelo seu módulo (magnitude)
    - Números negativos – representados pelo complemento para 2
    - Extensão para mais bits de representação é obtida por extensão do bit de sinal (bit mais à esquerda)
    - Uma única representação para o valor zero!

Decimal	Complemento para 2
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	---
-1	1111
-2	1110
-3	1101
-4	1100

bit de sinal

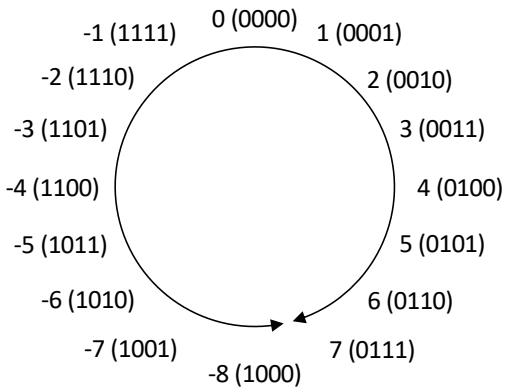
Complemento para 2

- Complemento para 2 (complemento verdadeiro)
  - Uma outra forma de determinar o complemento para 2 é manter inalterados todos os 0's menos significativos e o primeiro 1 menos significativo e inverter todos os restantes bits mais significativos
    - Exemplo: O complemento para 2 de $0010100_{(2)}$ é $1101100_{(2)}$
  - O complemento para 2 do complemento para 2 de um número x é o próprio número x:
    - $2^n - (2^n - x) = x$

Decimal	Complemento para 2
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	---
-1	1111
-2	1110
-3	1101
-4	1100

Complemento para 2

- Complemento para 2 (complemento verdadeiro)
  - Com n bits é possível representar 2^n números
    - Na representação em complemento para 2, representam-se os números no domínio $[-2^{n-1}, 2^{n-1} - 1]$
    - Exemplo: com 4 bits, é possível representar os números no domínio [-8, 7]



Decimal	Complemento para 2
+4	0100
+3	0011
+2	0010
+1	0001
+0	0000
-0	---
-1	1111
-2	1110
-3	1101
-4	1100

Representação de números inteiros (comparação)

- Números binários com sinal
  - As operações com representação em sinal e magnitude são mais complexas devido à necessidade de tratar o sinal e a magnitude em separado
  - A representação em complemento para 2 é a preferida por ter apenas uma representação para o valor 0 e por permitir operações simplificadas

Decimal	Sinal e magnitude $[-2^{n-1}-1, 2^{n-1}-1]$	Complemento para 1 $[-2^{n-1}-1, 2^{n-1}-1]$	Complemento para 2 $[-2^{n-1}, 2^{n-1}-1]$
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	1000	1111	--------
-1	1001	1110	1111
-2	1010	1101	1110
-3	1011	1100	1101
-4	1100	1011	1100

Complemento para 2 adição

- Adição de números representados em complemento para 2
  - A adição de números representados em complemento para 2 é realizada através da simples adição dos dois números incluindo os bits de sinal
  - Exemplos de adição para operandos com 4 bits:

4	0 1 0 0	4	0 1 0 0	4	0 1 0 0
+ 3	+ 0 0 1 1	+ (-3)	+ 1 1 0 1	+ (-4)	+ 1 1 0 0
7	0 1 1 1	1	0 0 0 1	0	0 0 0 0
3	0 0 1 1	3	0 0 1 1	3	0 0 1 1
+ 4	+ 0 1 0 0	+ (-4)	+ 1 1 0 0	+ (-3)	+ 1 1 0 1
7	0 1 1 1	-1	1 1 1 1	0	0 0 0 0

bit de sinal

Complemento para 2 subtração

- Subtração de números representados em complemento para 2
  - A subtração de números representados em complemento para 2 é realizada através da adição do diminuendo com o simétrico do diminuidor
    - $X - Y = X + (-Y) = X + \left(2^N - Y\right) = X + Y_{(C'2)}$
  - Exemplos de subtração para operandos com 4 bits:

4	0 1 0 0		4	0 1 0 0
- 3	- 0 0 1 1	=	+ (-3)	+ 1 1 0 1
1	0 0 0 1		1	0 0 0 1

bit de sinal

3	0 0 1 1		3	0 0 1 1
- 4	- 0 1 0 0	=	+ (-4)	+ 1 1 0 0
-1	(1) 1 1 1 1		-1	1 1 1 1

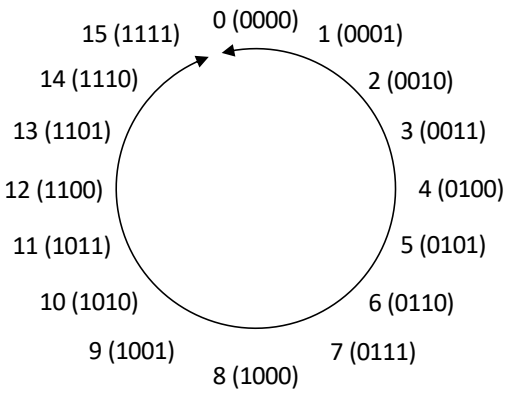
Capacidade de representação

- Como não é possível aumentar a capacidade de representação nos sistemas digitais, é necessário detetar as situações em que existe *overflow* (situação em que a capacidade de representação é excedida)
- Quando os números são naturais, o *Carry* (para a adição) ou o *Borrow* (para a subtracção) mais significativos assinalam se a capacidade de representação foi excedida (resultado fora de $[0, 2^n - 1]$)
- Para os números inteiros relativos representados em complemento para 2 deteta-se o *overflow* (resultado fora de $[-2^{n-1} - 1, 2^{n-1} - 1]$) usando outras condições lógicas

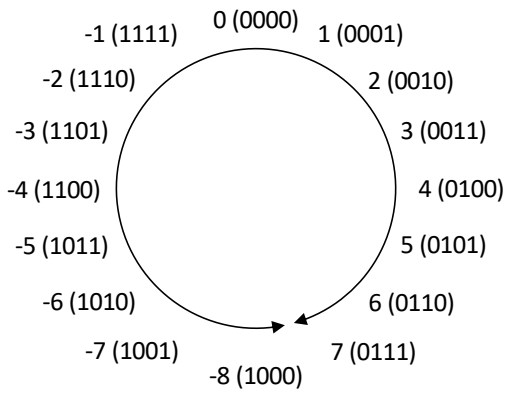
Overflow

- Capacidade de representação
  - Exemplo para números representados com 4 bits

Naturais



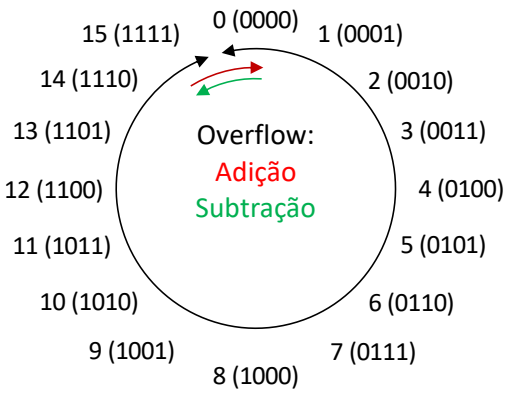
Relativos



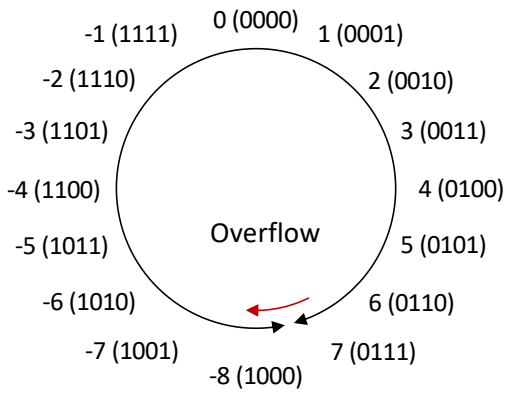
Overflow

- Capacidade de representação
  - Exemplo para números representados com 4 bits

Naturais
[0, 15]

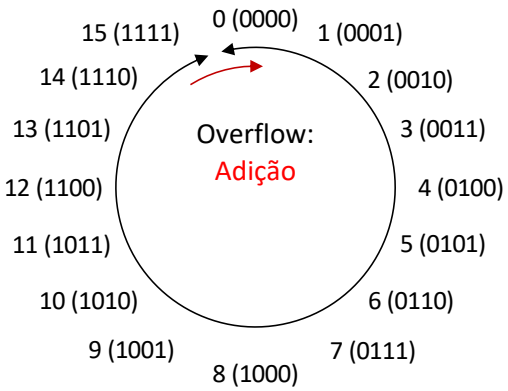


Relativos
[-8, 7]



Overflow para números naturais

- Deteção de overflow - Adição
  - Exemplo para números naturais com 4 bits



$$\begin{array}{rrrrr} & 0 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 0 \\ \hline (0) & 1 & 1 & 1 & 1 \end{array} \qquad \begin{array}{r} 3 \\ + \ 12 \\ \hline 15 \end{array}$$

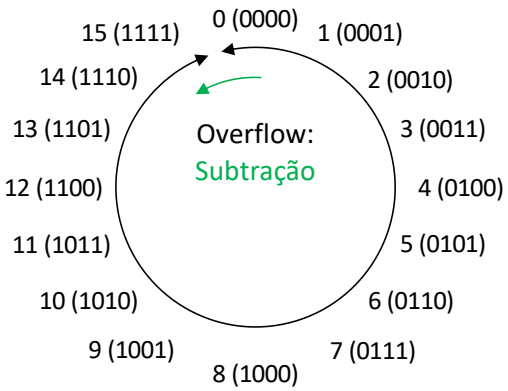
Contido no domínio [0, 15]
CarryOut = 0

$$\begin{array}{rrrrr} & 0 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline (1) & 0 & 0 & 0 & 0 \end{array} \qquad \begin{array}{r} 3 \\ + \ 13 \\ \hline (??)\ 0 \end{array}$$

Excedeu o domínio [0, 15]
CarryOut = 1

Overflow para números naturais

- Deteção de overflow – Subtração
  - Exemplo para números naturais com 4 bits



	1	0	1	0		10
-	0	0	1	1	-	3
(0)	0	1	1	1		7

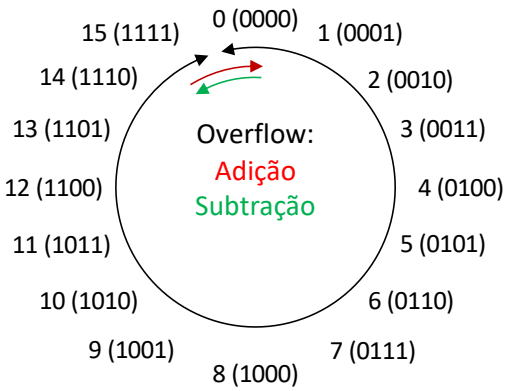
Contido no domínio [0, 15]
BorrowOut = 0

	0	0	1	1		3
-	0	1	0	1	-	5
(1)	1	1	1	0	(??)	14

Excedeu o domínio [0, 15]
BorrowOut = 1

Overflow para números naturais

- Deteção de overflow – Subtração (realizada através de adição)
  - $X - Y = X + (-Y) = X + (2^N - Y) = X + Y_{(C'2)}$ $\qquad$ $\text{BorrowOut} = \overline{\text{CarryOut}}$
  - Exemplo para números naturais com 4 bits



$10 - 3 = 10 + (2^4 - 3) = 10 + 13$

	1	0	1	0		10
+	1	1	0	1	+	13
(1)	0	1	1	1	(23 mod 16)	7

Contido no domínio [0, 15]
CarryOut = 1 => BorrowOut = 0

$3 - 5 = 3 + (2^4 - 5) = 3 + 11$

	0	0	1	1		3
+	1	0	1	1	+	11
(0)	1	1	1	0		14

Excedeu o domínio [0, 15]
CarryOut = 0 => BorrowOut = 1

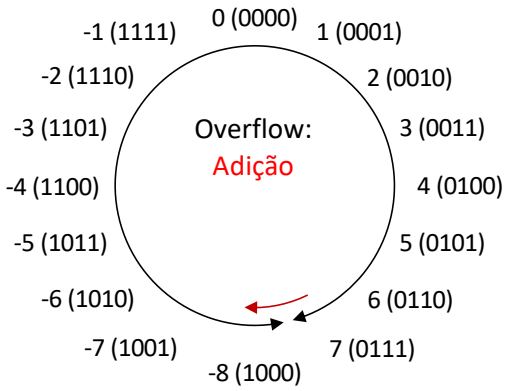
Overflow para números em complemento para 2

- O overflow para números em complemento para 2 pode ocorrer se forem somados:
  - dois operandos positivos

    ou
  - dois operandos negativos
- A condiçao de *overflow (OV)* pode ser detectada por inspecção dos sinais dos operandos e do resultado:
  - operandos positivos e resultado negativo => overflow
  - operandos negativos e resultado positivo => overflow
  - $OV = A_{n-1}.B_{n-1}.\overline{S_{n-1}} + \overline{A_{n-1}}.\overline{B_{n-1}}.S_{n-1}$

Overflow para números relativos (complemento para 2)

- Deteção de overflow
  - Exemplo para números em complemento para 2 com 4 bits



	0 0 1 1	3
+	0 0 1 0	+ 2
	0 1 0 1	5

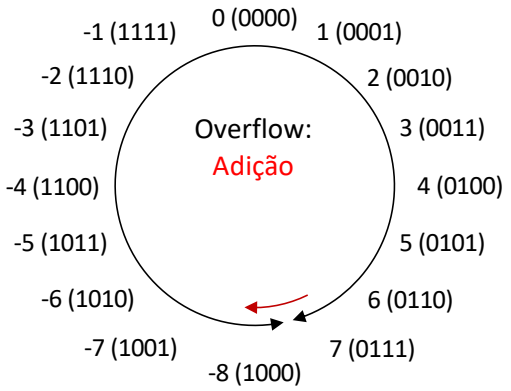
Contido no domínio [-8, 7]
Overflow = 0

	0 0 1 1	3
+	0 1 1 0	+ 6
	1 0 0 1	(??) -7

Excedeu o domínio [-8, 7]
Overflow = 1

Overflow para números relativos (complemento para 2)

- Deteção de overflow
  - Exemplo para números em complemento para 2 com 4 bits
  - Subtração é realizada à custa de somas: $X - Y = X + (-Y)$



6 – 5 = 6 + (-5)

	0	1	1	0	6
+	1	0	1	1	+ (-5)
	0	0	0	1	1

Contido no domínio [-8, 7]
Overflow = 0

– 5 – 6 = – 5 + (– 6)

	1	0	1	1	-5
+	1	0	1	0	+ -6
	0	1	0	1	(??) 5

Excedeu o domínio [-8, 7]
Overflow = 1

Complemento para 2
Overflow

- Exemplos

Soma de dois números positivos

$$\begin{array}{rl} 0\ 1\ 1\ 0 & +6_{(10)} \\ +\ 0\ 0\ 1\ 1 & +3_{(10)} \\ \hline \end{array}$$

Complemento para 2
Overflow

- Exemplos

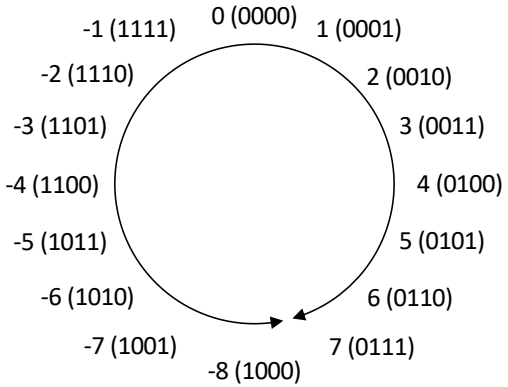
Soma de dois números positivos

$$\begin{array}{rl} 0\ 1\ 1\ 0 & +6_{(10)} \\ +\ 0\ 0\ 1\ 1 & +3_{(10)} \\ \hline 1\ 0\ 0\ 1 & \end{array}$$

Complemento para 2
Overflow

- Exemplos

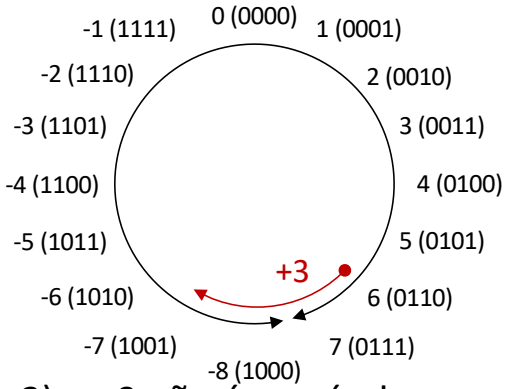
0 1 1 0	$+6_{(10)}$
+ 0 0 1 1	$+3_{(10)}$
1 0 0 1	$-7_{(10)}$????



Complemento para 2
Overflow

- Exemplos

0 1 1 0	$+6_{(10)}$
+ 0 0 1 1	$+3_{(10)}$
1 0 0 1	$-7_{(10)}$????



- Ocorreu *overflow* porque o valor (6+3) = +9 não é possível ser representado em código de complemento para 2 com 4 bits
  - Domínio complemento para 2 com 4 bits: [-8, 7]

Complemento para 2
Overflow

- Exemplos

Soma de dois números negativos

$$\begin{array}{rl} 1\ 1\ 0\ 0 & -4_{(10)} \\ +\ 1\ 0\ 1\ 0 & -6_{(10)} \\ \hline \end{array}$$

Complemento para 2
Overflow

- Exemplos

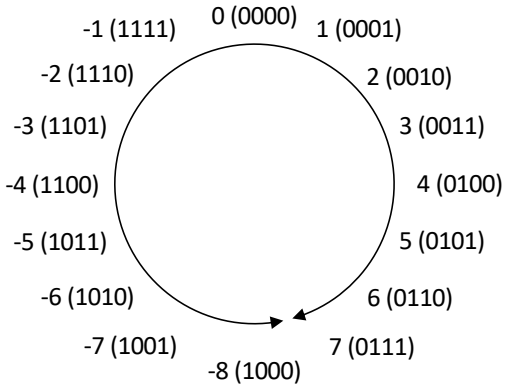
Soma de dois números negativos

$$\begin{array}{rl} 1\ 1\ 0\ 0 & -4_{(10)} \\ +\ 1\ 0\ 1\ 0 & -6_{(10)} \\ \hline 0\ 1\ 1\ 0 & \end{array}$$

Complemento para 2
Overflow

- Exemplos

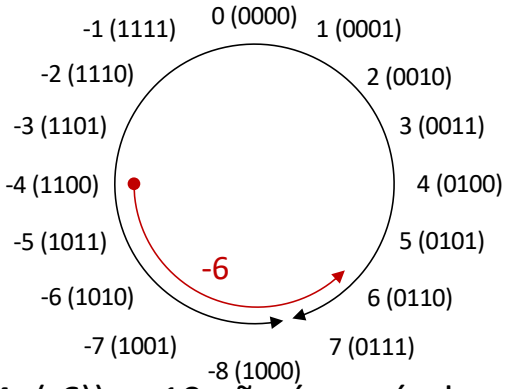
1 1 0 0	$-4_{(10)}$
+ 1 0 1 0	$-6_{(10)}$
0 1 1 0	$+6_{(10)}$????



Complemento para 2
Overflow

- Exemplos

1 1 0 0	$-4_{(10)}$
+ 1 0 1 0	$-6_{(10)}$
0 1 1 0	$+6_{(10)}$????



- Ocorreu *overflow* porque o valor (-4+(-6)) = -10 não é possível ser representado em código de complemento para 2 com 4 bits
  - Domínio complemento para 2 com 4 bits: [-8, 7]

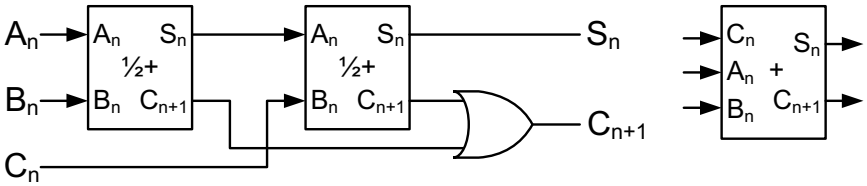
ISEL
DEETC
Departamento de
Engenharia Electrónica e
de Telecomunicações e
de Computadores

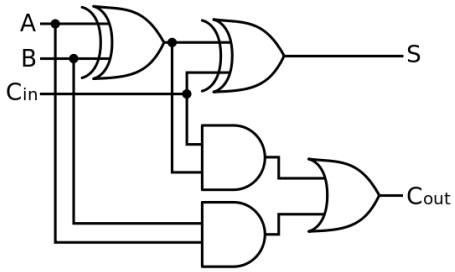
Circuitos aritméticos

(continuação)

Somador completo (full-adder)

- O circuito somador completo (*full-adder*) soma 3 bits de entrada (incluindo o transporte/*Carry* anterior) e produz 1 bit da soma e 1 bit de transporte

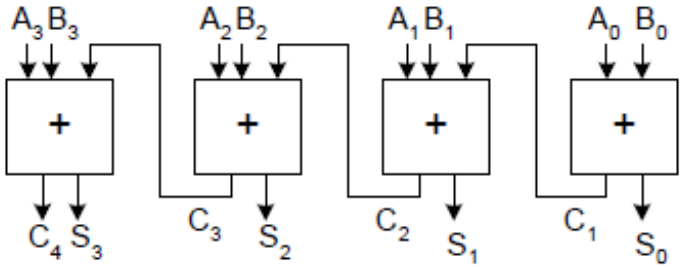




- $S_n = A_n \oplus B_n \oplus C_n$
- $C_{n+1} = A_n \,.\, B_n + (A_n \oplus B_n).\, C_n = A_n.\, B_n + A_n.\, C_n + B_n.\, C_n$

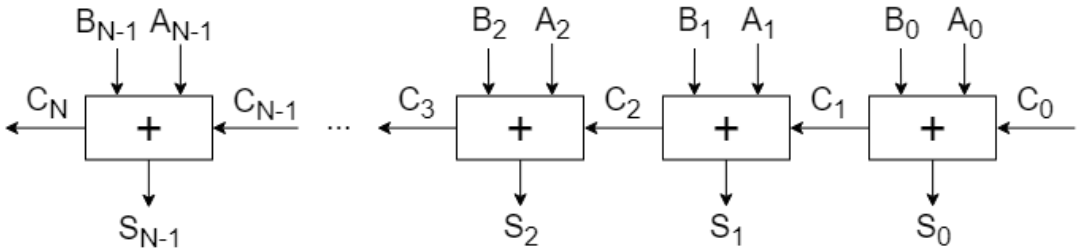
Somador de n bits

- O somador de *n* bits soma dois números (A e B) com *n* bits cada
- As saídas são:
  - Soma ($S_3..S_0$), igualmente com *n* bits
  - Carry (C_n)
- Solução iterativa:
  - Utiliza módulos replicáveis (*full-adders*)



Somador em cascata (*ripple carry adder*)

- Utiliza uma solução iterativa
- Desvantagem:
  - Tempo para o cálculo (i.e. o tempo de propagação) é grande
    - Deve-se à propagação do carry pelos módulos somadores (desde o 1º bit até ao último)
    - No pior caso é N * t_{pfa} (t_{pfa} = tempo propagação do *full adder*, 3 portas lógicas)
- É o somador que requer o menor número de portas lógicas



Circuito subtrator
Representação de números inteiros

- Considerando que representar em complemento para 2 é o mesmo que representar em complemento para 1 e adicionar 1, i.e:
  - $\boxed{(2^n) - N} = \boxed{[(2^n - 1) - N]} + 1$

complemento para 2 = (complemento para 1) + 1

(em base 2, o complemento para 1 equivale a inverter todos os bits)

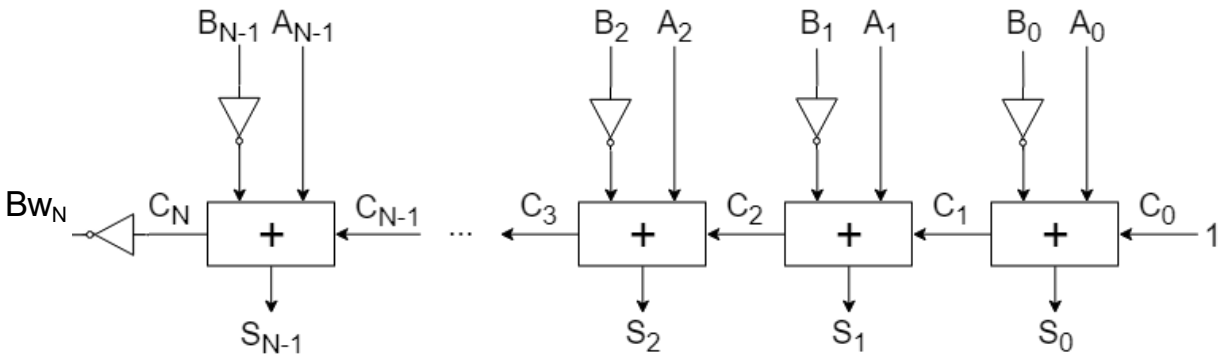
- A subtracção usando a representação em complemento para 2 pode ser obtida usando um somador onde o operando a subtrair tem os seus bits invertidos (complemento para 1) e a soma de 1 é obtida colocando o *Carry* de entrada a 1

Circuito subtractor

- Usando o circuito somador anteriormente apresentado, podemos alterá-lo para realizar a função de subtracção de números binários usando representação em complemento para 2

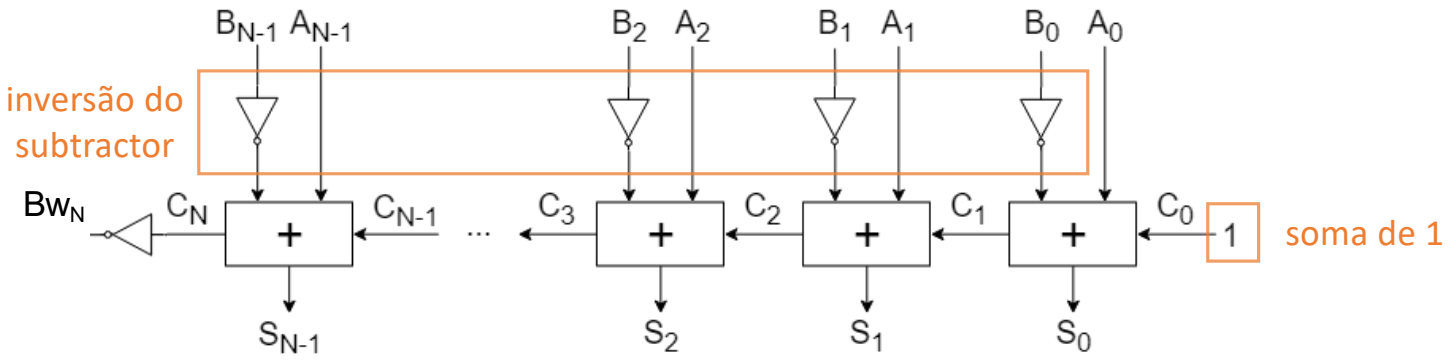
Circuito subtractor

- Usando o circuito somador anteriormente apresentado, podemos alterá-lo para realizar a função de subtracção de números binários usando representação em complemento para 2



Circuito subtractor

- Usando o circuito somador anteriormente apresentado, podemos alterá-lo para realizar a função de subtracção de números binários usando representação em complemento para 2



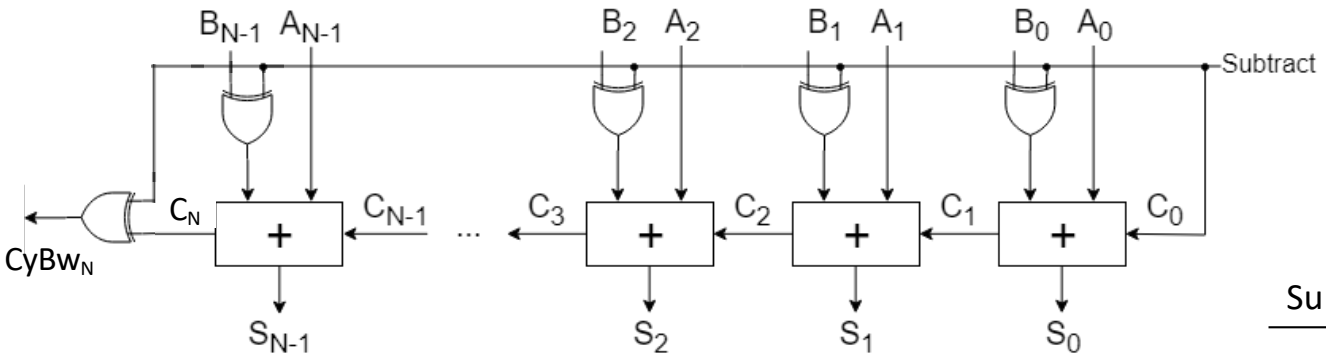
Circuito somador/subtractor

- Utilizando um inversor controlado (XOR), um único circuito pode ser usado para realizar as operações de soma e subtracção de números binários usando representação em complemento para 2

Subtract	Função
0	Adição
1	Subtracção

Circuito somador/subtractor

- Utilizando um inversor controlado (XOR), um único circuito pode ser usado para realizar as operações de soma e subtracção de números binários usando representação em complemento para 2



Subtract	Função
0	Adição
1	Subtracção

Overflow

- Para os números naturais deteta-se o *overflow* quando:
  - na operação de soma, o *Carry* mais significativo tiver o valor 1
  - na operação de subtração, o *Borrow* mais significativo tiver o valor 1

- Para os números inteiros relativos representados em complemento para 2 deteta-se o *overflow* se a operação de soma tiver:
  - operandos positivos e resultado negativo

  ou
  - operandos negativos e resultado positivo

Overflow em complemento para 2

- As condições lógicas para detecção de *overflow* (OV) na soma de 2 operandos com *n* bits é dada pela expressão:
  - $OV = A_{n-1}.B_{n-1}.\overline{S_{n-1}} + \overline{A_{n-1}}.\overline{B_{n-1}}.S_{n-1}$
- Dada a função lógica do circuito somador completo, podemos deduzir que a expressão anterior é equivalente a:
  - $OV = A_{n-1}.B_{n-1}.\overline{C_{n-1}} + \overline{A_{n-1}}.\overline{B_{n-1}}.C_{n-1}$
    - Para A_{n-1}=1 e B_{n-1}=1, S_{n-1}=0 sse C_{n-1}=0; Para A_{n-1}=0 e B_{n-1}=0, S_{n-1}=1 sse C_{n-1}=1
  - $OV = C_n.\overline{C_{n-1}} + \overline{C_n}.C_{n-1}$
    - Quando A_{n-1}=1 e B_{n-1}=1, C_n=1; Quando A_{n-1}=0 e B_{n-1}=0, C_n=0
  - $OV = C_n \oplus C_{n-1}$

		n	n-1	n-2
carry		1	0	-
A			1	-
B	+		1	-
S			0	-

		n	n-1	n-2
carry		0	1	-
A			0	-
B	+		0	-
S			1	-

Diferença entre Carry e Overflow

Representação	**CarryBorrow** **$CyBw_n = 1$**	**OV = 1**
Naturais (sem sinal)	Capacidade de representação excedida	Sem significado
Complemento para 2 (com sinal)	Sem significado	Capacidade de representação excedida

Temas abordados

Sumário

- Representação de números inteiros
  - Complemento para 2
- Excesso de capacidade de representação (Overflow)

Próxima aula

- Resolução de exercícios